

CODEBASE DOCUMENTATION

SolarExpress

Technical Architecture — Block-by-Block Reference

Document version: 1.0

Date: 6 August 2026

Scope: Full repository — Solidity, Node/Express backend, Next.js frontend, operational scripts

System	Interplanetary travel booking with ERC-721 NFT tickets
Network	Ethereum Sepolia testnet (chain id 11155111)
Frontend	Next.js 16 (App Router) / React 19 / TypeScript / Tailwind v4 / React Three Fiber
Backend	Node.js / Express 4 / MongoDB (Mongoose) / JWT
Blockchain tooling	Wagmi 3 / Viem 2 / Solidity 0.8.27 / OpenZeppelin 5.4
Storage	MongoDB (bookings), IPFS/Pinata (NFT images & metadata)

Prepared from the current working tree of the SolarExpress repository.

Table of Contents

Part A — System Overview

- A.1 Executive Summary
- A.2 System Architecture
- A.3 Deployment Topology
- A.4 Technology Stack
- A.5 Environment Variables
- A.6 Deployed Contracts & Accounts
- A.7 End-to-End Booking Flow
- A.8 Repository Structure
- A.9 Document Conventions

Part B — Smart Contracts (Solidity)

- B.1 TicketSale.sol
- B.2 SolarExpressTicket.sol
- B.3 BookingHistory.sol

Part C — Backend (Node/Express)

- C.1 server.js
- C.2 config/database.js
- C.3 models/User.js
- C.4 models/Booking.js
- C.5 models/Notification.js
- C.6 middleware/requireAuth.js
- C.7 Routes (5 files)
- C.8 controllers/authController.js
- C.9 controllers/userController.js
- C.10 controllers/bookingController.js
- C.11 controllers/notificationController.js
- C.12 controllers/nftController.js
- C.13 services/verifyPurchase.js
- C.14 services/pinataService.js
- C.15 utils/generateFlightDetails.js
- C.16 scripts/backfillFromContract.js
- C.17 scripts/backfillFlightDetails.js
- C.18 backend/package.json

Part D — Frontend (Next.js)

- D.1 Root Configuration
- D.2 App Shell (layout, globals, loading)
- D.3 Providers, Context & Client Libs
- D.4 API, ABI & NFT Libs
- D.5 Hooks
- D.6 Static Data (bodies, travel, routes, art)

D.7 App Pages

D.8 API Routes (NFT metadata/image pipeline)

D.9 UI & Layout Components

D.10 3D Scene Components

D.11 Panel & Menu Components

D.12 NFT Visual Components

Part E — Appendices

E.1 Operational Scripts

E.2 Repository README

E.3 Known Limitations & Pending Actions

Part A — System Overview

A.1 Executive Summary

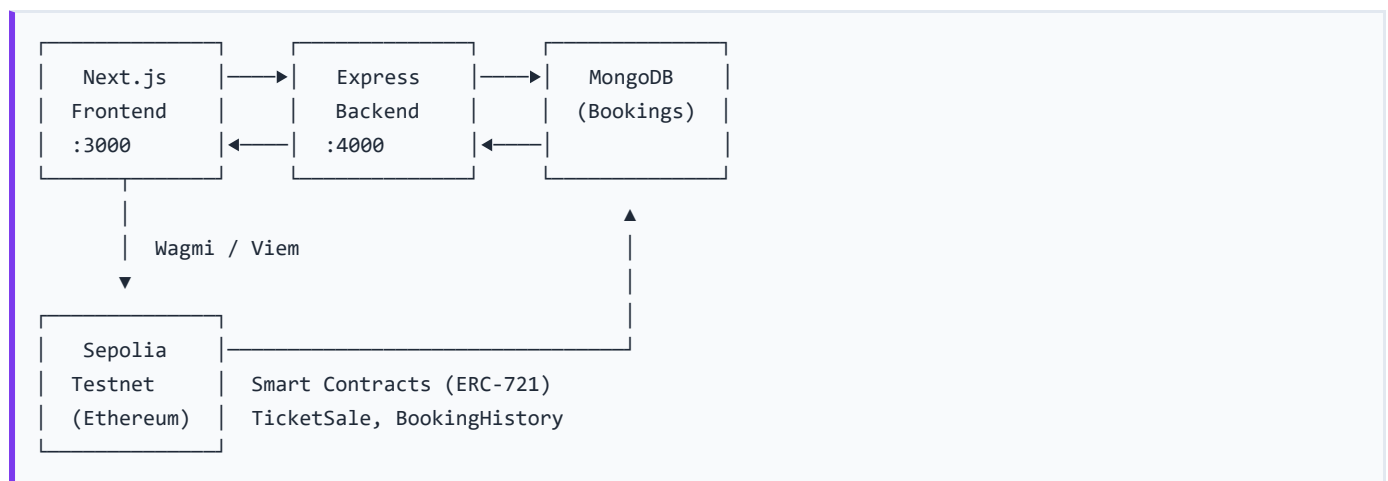
SolarExpress is an interplanetary travel booking platform that combines a modern Next.js web application with blockchain technology. Users browse an interactive 3D solar system, book flights to 20 destinations (planets and moons), pay in Sepolia test ETH, and receive a unique ERC-721 NFT boarding pass. Each purchase is verified on-chain by the backend, recorded in MongoDB, and made observable to the wallet owner via an on-chain booking history contract. NFT images and metadata are generated procedurally, uploaded to IPFS via Pinata, and pointed to by the token URI.

The system is divided into four cooperating parts:

- **Three Solidity contracts** at the repository root: `TicketSale.sol` (pricing & payment), `SolarExpressTicket.sol` (the ERC-721 NFT), and `BookingHistory.sol` (per-user on-chain history).
- **An Express backend** (`backend/`) providing JWT authentication, booking persistence, on-chain purchase verification, Pinata/IPFS upload, and notifications.
- **A Next.js frontend** (`src/`) providing the App-Router pages, the React Three Fiber solar system, the booking wizard, the NFT gallery, and server-side NFT metadata/image routes.
- **Operational scripts** (`scripts/` , `backend/scripts/`) for IPFS migration, contract wiring, one-off backfills, and test mints.

A.2 System Architecture

The diagram below (reproduced from `README.md`) summarises the principal data paths:



Key architectural decisions:

- **Separation of concerns on-chain.** Pricing/payment, token ownership, and booking history live in three distinct contracts. `TicketSale` is the only contract that can mint tickets or record bookings (`onlySaleContract` guards), which keeps history accurate even if an NFT later changes hands.
- **Hybrid booking record.** The source of truth for "who paid" is the chain; MongoDB stores the enriched record (flight number, seat, terminal, reference). The backend refuses to persist a booking until the transaction receipt is verified on-chain.
- **Decentralised ticket media.** Each newly minted token gets a per-token immutable IPFS metadata URI (`setTokenURI`), so Etherscan/marketplaces resolve artwork without a live host. Mints that are not performed by the contract owner fall back to an on-demand `/api/nft/metadata/[tokenId]` route backed by `baseTokenURI` .

- **Cookie-based auth.** JWT is delivered in an `httpOnly` cookie (with a Bearer-header fallback for mobile/programmatic clients).

A.3 Deployment Topology

Component	Host	Notes
Next.js frontend	Vercel	App Router; API rewrites proxy <code>/api/auth users bookings notifications/:path*</code> to the backend URL; NFT metadata/image routes run as serverless functions.
Express backend	Render (proxy verified)	Node <code>server.js</code> on port 4000; serves auth, booking, notification and Pinata routes.
MongoDB	MongoDB Atlas	Connection string in <code>MONGODB_URI</code> ; <code>users</code> , <code>bookings</code> , <code>notifications</code> collections.
Smart contracts	Ethereum Sepolia	Three deployed contracts (see A.6); owner/deployer key held by the project.
IPFS	Pinata	Pinned ticket PNGs + metadata JSON; <code>PINATA_JWT</code> required for the mint pipeline.

A.4 Technology Stack

Layer	Technologies
Frontend	Next.js 16 (App Router), React 19, TypeScript (strict), Tailwind CSS v4, Wagmi 3 / Viem 2, TanStack Query 5, React Three Fiber 9, React Three Drei 10, Three.js 0.185, <code>@resvg/resvg-js</code> (server SVG→PNG).
Backend	Node.js, Express 4.21, Mongoose 9.8, jsonwebtoken, bcryptjs, helmet, cookie-parser, cors, express-rate-limit, sharp, dotenv.
Blockchain	Solidity ^0.8.27, OpenZeppelin 5.4 (<code>Ownable</code> , <code>ReentrancyGuard</code> , <code>ERC721</code> , <code>ERC721URIStorage</code>).
Tooling	npm, Node 18+, Vercel CLI, no Hardhat/Foundry project present (contracts are compiled/deployed outside this repo).

A.5 Environment Variables

Variable names are documented below. Values (tokens, keys, connection strings) are intentionally omitted from this document for security.

Frontend — `.env.local` (root)

Variable	Purpose
<code>NEXT_PUBLIC_TICKET_SALE_ADDRESS</code>	Deployed <code>TicketSale</code> contract address (overrides <code>src/lib/contract.ts</code> default).
<code>NEXT_PUBLIC_TICKET_NFT_ADDRESS</code>	Deployed <code>SolarExpressTicket</code> address.
<code>NEXT_PUBLIC_BOOKING_HISTORY_ADDRESS</code>	Deployed <code>BookingHistory</code> address.
<code>NEXT_PUBLIC_SEPOLIA_RPC_URL</code>	Optional Sepolia RPC; appended to the built-in fallback list in <code>src/lib/rpc.ts</code> .
<code>API_BACKEND_URL</code>	Backend base URL used by <code>src/lib/backend-url.ts</code> , API routes and rewrites.
<code>PINATA_JWT</code>	Pinata JWT for the IPFS mint pipeline (server-side, set in Vercel).

Note: the checked-in `.env.local` currently contains a short-lived Vercel OIDC token. It is ignored by git (`.env*`) and is not reproduced here.

Backend — `backend/.env`

Variable	Purpose
<code>MONGODB_URI</code>	MongoDB connection string.
<code>JWT_SECRET</code>	HMAC secret for signing JWTs; the middleware fails fast at import if unset.
<code>PORT</code>	HTTP port (default <code>4000</code>).
<code>CORS_ORIGIN</code>	Allowed CORS origin (default <code>http://localhost:3000</code>).
<code>NODE_ENV</code>	Drives cookie <code>secure</code> / <code>sameSite</code> behaviour.
<code>SALE_CONTRACT_ADDRESS</code>	Sale contract for <code>verifyPurchase</code> (default live address).
<code>PINATA_JWT</code>	Pinata JWT for <code>pinataService.js</code> .
<code>SEPOLIA_RPC_URL</code>	Optional first RPC in the verification fallback list.
<code>IPFS_GATEWAY</code>	Gateway base for image URLs (default <code>https://ipfs.io</code>).
<code>DISABLE_BOOKING_VERIFY</code>	Set <code>true</code> to skip on-chain verification in <code>createBooking</code> (dev only).
<code>TICKET_SALE_ADDRESS</code> / <code>NEXT_PUBLIC_TICKET_SALE_ADDRESS</code>	Backfill overrides for the sale contract address.
<code>START_BLOCK</code>	Backfill start block (optional; defaults to <code>latest - 200000</code>).

A.6 Deployed Contracts & Accounts

Contract	Address (Sepolia)	Role
<code>SolarExpressTicket</code> (NFT)	<code>0x76062Ce48e3fA5F24375A27A5Da8ea1d52ED2bee</code>	ERC-721 ticket token
<code>TicketSale</code>	<code>0xA3E410c1A85Ae21774a6aF2D54a818BaedF19eCE</code>	Pricing, payment, minting, booking recording
<code>BookingHistory</code>	<code>0xD56923c65aA2174728cF5b66D305948181cEf6bc</code>	Per-user on-chain history
Owner / deployer	<code>0x79C85bA709535F817A04FefF49e3d4b8f2648372</code>	<code>onlyOwner</code> privileged account

- On-chain `baseTokenURI` was updated to `https://solar-express.vercel.app/api/nft/metadata/` (the source default still reads `https://solar-express-robinson11.vercel.app/api/nft/metadata/`).
- Tickets 1–6 were retro-migrated to immutable IPFS metadata; the Pinata JWT currently in use expires around 2027.
- The deployer private key was exposed during the session; key rotation is a pending, user-side action.

A.7 End-to-End Booking Flow

- 1 User connects a wallet (Wagmi) on `/` ; `useBuyTicket` reads the destination price and opens the 9-step wizard (`BookTicketButton`).
- 2 User confirms; `buyTicket(destinationId, travelClass)` sends ETH to `TicketSale.buyTicket` . The contract checks `msg.value == price` , mints via `SolarExpressTicket.mintTicket` , records the trip in `BookingHistory.recordBooking` , and emits `TicketPurchased` .
- 3 The frontend stores the pending tx hash in `sessionStorage` (`solarexpress_pending_tx`) and waits for the receipt; the `TicketPurchased` log is decoded to obtain `tokenId` .

- 4 The frontend calls `/api/nft/generate-metadata`, which predicts the token id from recent `TicketPurchased` logs, renders the ticket SVG to PNG (resvg + bundled Roboto), uploads PNG and metadata JSON to Pinata, and (if the signer is the owner) sets the per-token IPFS URI via `setTokenURI`. The gateway image URL is warmed into the browser cache.
- 5 The frontend calls the backend `POST /api/bookings` with the receipt fields. `bookingController.createBooking` validates ownership, verifies the receipt on-chain (receipt status, event log, buyer/tokenId/destination/price match), rejects duplicates, fills any missing flight fields, and stores the enriched booking in MongoDB.
- 6 The user's `tickets` array is appended to and a `transaction` notification is created. The booking appears in `/booking-history` (NFT gallery + history tabs) using both MongoDB records and `BookingHistory.getBookings(address)`.

A.8 Repository Structure

```
SolarExpress/
├── TicketSale.sol           # Pricing + payment contract (root)
├── SolarExpressTicket.sol   # ERC-721 ticket contract (root)
├── BookingHistory.sol       # On-chain booking history contract (root)
├── backend/
│   ├── server.js           # Express bootstrap + route mounts
│   ├── config/database.js  # Mongoose connection + index hygiene
│   ├── controllers/        # auth, user, booking, notification, nft
│   ├── middleware/requireAuth.js
│   ├── models/             # User, Booking, Notification
│   ├── routes/             # auth, user, booking, notification, nft
│   ├── scripts/            # backfillFromContract, backfillFlightDetails
│   ├── services/           # verifyPurchase, pinataService
│   ├── utils/generateFlightDetails.js
│   └── package.json
├── scripts/                # ipfs-migrate.mjs, wire-with-viem.js,
                           # set-token-uri.js, mint-test-ticket.js
├── src/
│   ├── app/                # pages, globals.css, loading, api/nft/...
│   ├── components/         # layout/, menu/, nft/, panel/, scene/, ui/, wallet/
│   ├── context/            # auth, loading, notification providers
│   ├── data/               # bodies, travel, routes, nft-art, textures
│   ├── hooks/              # useBuyTicket, useTicketInfo, useSyncBooking, ...
│   └── lib/                 # api, contract, rpc, nft-service, ...
├── public/fonts/           # Roboto-Regular.ttf, Roboto-Bold.ttf
├── next.config.ts, tsconfig.json, package.json
└── README.md
```

A.9 Document Conventions

- Every source file is documented file-by-file; within each file, code is presented **block-by-block in source order** with the exact line ranges of the current working tree.
- Block titles name the symbols defined. Behaviour, invariants and security-relevant remarks are listed per block.
- "Live value" notes refer to the on-chain or deployed state; source defaults are quoted verbatim.
- Secrets and private key material are never reproduced.

Part B — Smart Contracts (Solidity)

The three contracts are deployed on Sepolia and compiled with Solidity ^0.8.27 against OpenZeppelin ^5.4.0. They are stored at the repository root rather than in a `contracts/` subfolder (the README's folder listing is stale in that respect).

B.1 TicketSale.sol (86 lines)

Handles pricing and ETH payment, mints the NFT via `SolarExpressTicket`, then records the trip in `BookingHistory`. This is the one contract that ties all three pieces together.

Block 1 · SPDX header & pragma (lines 1–3)

SPDX `UNLICENSED`, a compatibility note for OpenZeppelin ^5.4.0, and `pragma solidity ^0.8.27`. Establishes the compiler floor; `^` permits later 0.8.x releases.

Block 2 · Imports (lines 5–6)

`Ownable` (owner-restricted admin functions) and `ReentrancyGuard` (mutex around the payable entry point).

Block 3 · Interface `ISolarExpressTicket` (lines 8–13)

Declares `mintTicket(address to, uint256 destinationId) external returns (uint256)` — the only interaction with the NFT contract. Used to type the immutable reference without importing the full contract.

Block 4 · Interface `IBookingHistory` (lines 15–22)

Declares `recordBooking(address user, uint256 ticketId, uint256 destinationId, uint256 pricePaid)` — the write path into the history contract.

Block 5 · Contract declaration (lines 24–27)

Natspec states the responsibilities (pricing, payment, mint, record). Contract inherits `Ownable`, `ReentrancyGuard`.

Block 6 · `enum TravelClass` (line 28)

Three travel classes: `Economy`, `Business`, `First` — used as the second key of the price map.

Block 7 · Immutable contract references (lines 30–31)

`ticketContract` and `bookingHistory` are `immutable` — fixed at construction, gas-efficient, and guarantee the wiring cannot be changed afterwards.

Block 8 · Price table (lines 33–34)

`mapping(uint256 => mapping(TravelClass => uint256)) public destinationPrice` maps `destinationId` → `travelClass` → `priceWei`. A zero price means the destination/class is not bookable.

Block 9 · Event `TicketPurchased` (lines 36–41)

Four fields, with `tokenId`, `buyer` and `destinationId` indexed. This event is the canonical on-chain proof of purchase used by the backend (`verifyPurchase`), the backfill script, and the token-id predictor.

Block 10 · Constructor (lines 43–48)

Takes the deployed addresses of `SolarExpressTicket` and `BookingHistory`, wraps them in the interfaces, and passes `msg.sender` to `Ownable`. Deployer becomes contract owner.

Block 11 · `setDestinationPrice` (lines 50–52)

`onlyOwner`. Sets a single `(destinationId, travelClass)` price in wei.

Block 12 · `setDestinationPrices` (lines 54–63)

`onlyOwner` . Bulk variant: requires equal-length `destinationIds` and `pricesWei` arrays, then writes each pair for one `travelClass` . Used by the test-mint script to configure all 20 destinations.

Block 13 · `buyTicket` (lines 65–80)

`payable` and `nonReentrant` . Reads the price and requires it is non-zero; requires `msg.value == price` (exact payment, no change). Calls `mintTicket` on the NFT contract, records the booking on-chain, emits `TicketPurchased` , and returns the new `tokenId` . The `nonReentrant` modifier prevents re-entrancy into the ETH-handling path; the contract itself never holds funds beyond the purchase (see `withdraw`).

Block 14 · `withdraw` (lines 82–85)

`onlyOwner` . Sends the whole balance to the owner with a low-level `call` and reverts on failure. Because `buyTicket` requires exact ETH and immediately mints/records, the balance is normally zero.

B.2 `SolarExpressTicket.sol` (80 lines)

Pure NFT contract. Holds no pricing/payment logic; only the approved `TicketSale` contract (or the owner, for setup) can mint.

Block 1 · SPDX header & pragma (lines 1–3)

Same header convention as `TicketSale.sol` .

Block 2 · Imports (lines 5–8)

`ERC721` , `ERC721URIStorage` (per-token URI + `MetadataUpdate` signalling), `Ownable` , and `Strings` (uint256→string concatenation).

Block 3 · Contract declaration (lines 10–13)

Inherits `ERC721` , `ERC721URIStorage` , `Ownable` ; natspec notes the separation of concerns from `TicketSale` .

Block 4 · Counter & base URI (lines 14–15)

`_nextTokenId` starts at 1 (token 0 never mints). `baseTokenURI` default is `https://solar-express-robinson11.vercel.app/api/nft/metadata/` . **Live value** was updated on-chain to `https://solar-express.vercel.app/api/nft/metadata/` .

Block 5 · `saleContract` (lines 17–18)

The single address permitted to mint; set once after both contracts are deployed.

Block 6 · `TicketData` struct & `tickets` mapping (lines 20–25)

Stores per-token `destinationId` and `timestamp` (block time of mint); exposed via the public `tickets` mapping. This is how the metadata route and frontend resolve a token's destination on-chain.

Block 7 · Event `TicketMinted` (line 27)

`tokenId` , `buyer` , `destinationId` all indexed.

Block 8 · Constructor (line 29)

Deploys the token with name `SolarExpress Ticket` and symbol `SXT` ; deployer becomes owner.

Block 9 · Modifier `onlySaleContract` (lines 31–34)

Restricts the mint path to the configured sale contract.

Block 10 · `setSaleContract` (lines 36–40)

`onlyOwner` . Points `saleContract` at the deployed `TicketSale` ; call once after deployment.

Block 11 · `setBaseTokenURI` (lines 42–45)

`onlyOwner`. Changes the fallback base URI for tokens minted without an explicit IPFS URI.

Block 12 · `mintTicket` (lines 47–62)

`onlySaleContract`. Allocates `tokenId = _nextTokenId++`, `_safeMint`s to the buyer, sets the initial URI to `baseTokenURI + tokenId`, stores `TicketData` with `block.timestamp`, emits `TicketMinted`, and returns the id. `_safeMint` guards against minting to contract addresses that reject ERC-721.

Block 13 · `setTokenURI` (lines 64–70)

`onlyOwner`. Points an existing token at a new metadata URI (e.g. an IPFS CID) and emits ERC-4906 `MetadataUpdate` so marketplaces refresh. This is the function the IPFS migration and mint pipelines call — hence the owner-wallet signing requirement.

Block 14 · Overrides (lines 72–79)

`tokenURI` and `supportsInterface` disambiguate the two inherited implementations (`ERC721` / `ERC721URIStorage`) as required by the OZ extension.

B.3 `BookingHistory.sol` (76 lines)

Records a persistent, queryable booking history per user, separate from the NFT contract, so history stays accurate even if a ticket is later transferred or resold.

Block 1 · SPDX header & pragma (lines 1–3)

Standard header.

Block 2 · Import (line 5)

`Ownable` only — no token standard needed.

Block 3 · Contract declaration (lines 7–12)

Natspec explains the design rationale: booking facts are immutable history tied to the purchasing address, not to current NFT ownership.

Block 4 · `Booking` struct (lines 13–18)

Fields `ticketId` (equals the minted tokenId), `destinationId`, `pricePaid`, `timestamp`.

Block 5 · `saleContract` (lines 20–21)

The one contract allowed to record bookings.

Block 6 · State (lines 23–24)

`_bookingsByUser: mapping(address => Booking[])` (private) and `tripCount: mapping(address => uint256)` (public) for O(1) trip totals.

Block 7 · Event `BookingRecorded` (lines 26–32)

`user`, `ticketId`, `destinationId` indexed; also emits `pricePaid` and `timestamp`.

Block 8 · Constructor (line 34)

Sets deployer as owner.

Block 9 · Modifier `onlySaleContract` (lines 36–39)

Restricts writes to the sale contract.

Block 10 · `setSaleContract` (lines 41–44)

`onlyOwner`. Wires the sale contract address; call once after deployment.

Block 11 · `recordBooking` (lines 46–63)

`onlySaleContract` . Pushes a `Booking` , increments `tripCount` , emits `BookingRecorded` .

Block 12 · `getBookings` (lines 65–68)

`view` ; returns the full array for a user. Used by `useOnChainBookings` on the frontend.

Block 13 · `getBooking` (lines 70–75)

`view` ; returns one booking by index with an explicit out-of-range revert — cheaper than returning the whole array.

Part C — Backend (Node/Express)

C.1 backend/server.js (38 lines)

Express application entry point: middleware, route mounts, health check, and deferred startup after the DB connects.

Block 1 · Requires & env bootstrap (lines 1–11)

Loads `dotenv`, Express, cookie-parser, cors, helmet, then the database connection and the five route routers. Routers are imported before the app is configured.

Block 2 · App & constants (lines 13–15)

`PORT` from `PORT` env (default `4000`); `CORS_ORIGIN` from env (default `http://localhost:3000`).

Block 3 · Middleware (lines 17–21)

`app.set('trust proxy', 1)` (correct client IP behind Render's proxy, required for rate limiting); `helmet()` security headers; `cookieParser()`; CORS with `credentials: true` pinned to `CORS_ORIGIN`; JSON body parsing capped at `512kb`.

Block 4 · Route mounts (lines 23–28)

Mounts: `/api/auth` → `authRoutes`; `/api/users` → `userRoutes`; `/api` → `bookingRoutes` (`POST /bookings`, `GET /bookings/history`); `/api/notifications` → `notificationRoutes`; `/api` → `nftRoutes` (`POST /pinata/upload-ticket`).

Block 5 · Health check (lines 30–32)

`GET /api/health` returns `{ status: 'ok' }` — the endpoint used for proxy liveness checks.

Block 6 · Boot (lines 34–38)

Waits for `connectDB()` to resolve, then `app.listen(PORT)`. The server does not accept connections until MongoDB is reachable.

C.2 backend/config/database.js (20 lines)

Mongoose bootstrap with legacy-index cleanup.

Block 1 · Import (line 1)

`mongoose`.

Block 2 · connectDB (lines 3–18)

Connects with `process.env.MONGODB_URI`. On success, attempts to drop the legacy unique index `walletAddress_1` on `users` (previously forced one wallet per account); failure is swallowed because the index may not exist. On connection error, logs and `process.exit(1)`.

Block 3 · Export (line 20)

Default export of `connectDB`.

C.3 backend/models/User.js (64 lines)

Account model. Passwords stored hashed; a safe serialiser strips secret material.

Block 1 · Import (line 1)

`mongoose`.

Block 2 · Schema — identity fields (lines 3–37)

`email` required/unique/lowercased/trimmed with format regex; `passwordHash` required with `select: false` (never returned by default queries); `username` optional trim; `walletAddress` optional, lowercased; `profileImage` optional; `role`

```
enum user|admin default user.
```

Block 3 · Schema — embedded data (lines 38–50)

`tickets` sub-documents `{ destinationId, txHash, tokenId, purchasedAt }` (a denormalised mirror of on-chain purchases) and `favorites` array of destination ids.

Block 4 · Schema options (lines 52–54)

```
timestamps: true adds createdAt / updatedAt.
```

Block 5 · `toSafeObject` method (lines 57–62)

Returns a plain object with `passwordHash` and `__v` deleted — the safe serialisation used by auth responses.

Block 6 · Model export (line 64)

```
mongoose.model('User', userSchema).
```

C.4 backend/models/Booking.js (67 lines)

Enriched booking record; uniqueness and enumeration constraints prevent duplicates and invalid states.

Block 1 · Import (line 1)

```
mongoose.
```

Block 2 · Schema — core fields (lines 3–35)

`userId` (ObjectId ref `User`, required); `walletAddress` required/lowercased; `destinationId` required Number; `transactionHash` required, **unique**, lowercased (primary dedupe key); `tokenId` required and indexed; `pricePaid` stored as String (ETH decimal form); `status` enum `Confirmed|Refunded|Cancelled` default `Confirmed`.

Block 3 · Schema — flight enrichment (lines 41–60)

`bookingReference` unique with `sparse: true` (optional but unique when present); `departureDate`, `departureTime`; `travelClass` enum `economy|business|first` default `economy`; `seatNumber`; `availabilityStatus` default `pending`; `availabilityCheckedAt`; `flightNumber`; `launchTerminal`.

Block 4 · Options & export (lines 62–67)

```
timestamps: true; model name Booking.
```

C.5 backend/models/Notification.js (13 lines)

In-app notifications for a user.

Block 1 · Schema & export (lines 1–13)

Single block: `userId` (ref `User`, required); `type` enum `welcome|promotion|transaction` required; `message` required; `read` and `dismissed` booleans default false; optional `tokenId` and `txHash` for transaction notifications; `timestamps: true`. Exported as `Notification`.

C.6 backend/middleware/requireAuth.js (34 lines)

Shared JWT guard used by protected routes; also the single source of `JWT_SECRET`.

Block 1 · Imports (lines 1–2)

`jsonwebtoken` and the `User` model.

Block 2 · Fail-fast secret check (lines 4–8)

Throws at module load if `JWT_SECRET` is unset, preventing a silently mis-secured deployment. Exports the constant for use

by the auth controller.

Block 3 · `requireAuth` middleware (lines 10–32)

Accepts the token from the `token` cookie (browsers) or a `Bearer` Authorization header (mobile/programmatic). 401 if absent. Verifies the JWT, loads the user (`select` projection), 401 if the user was deleted, and attaches `req.user = { id, email, walletAddress }`. Any verification failure yields a generic `Invalid or expired token` 401.

Block 4 · Exports (line 34)

```
{ requireAuth, JWT_SECRET }.
```

C.7 Routes (5 files)

Thin routers; all business logic lives in controllers.

`authRoutes.js` (21 lines)

Imports the auth controller and a rate limiter. `authLimiter` (lines 8–14) allows `20` requests per `15 minutes` with `standardHeaders: true` and `legacyHeaders: false`. Routes: `POST /signup` (limited), `POST /login` (limited), `POST /logout`, `GET /me` (protected).

`userRoutes.js` (10 lines)

Both routes protected: `PATCH /wallet` → `linkWallet`; `DELETE /wallet` → `unlinkWallet`.

`bookingRoutes.js` (10 lines)

Both protected: `POST /bookings` → `createBooking`; `GET /bookings/history` → `getBookingHistory`. Mounted at `/api`, so full paths are `/api/bookings` and `/api/bookings/history`.

`notificationRoutes.js` (14 lines)

Applies `requireAuth` router-wide (line 6). `GET / list`; `PATCH /:id/read`; `PATCH /read-all`; `PATCH /:id/dismiss`; `POST / create`.

`nftRoutes.js` (8 lines)

No auth. `POST /pinata/upload-ticket` → `uploadTicket` (mounted at `/api`, full path `/api/pinata/upload-ticket`).

C.8 backend/controllers/authController.js (115 lines)

Signup/login/logout/me with bcrypt hashing and httpOnly cookie JWT issuance.

Block 1 · Imports & secret (lines 1–5)

`bcryptjs`, `jsonwebtoken`, `User`, `Notification`, and `JWT_SECRET` from the middleware.

Block 2 · Cookie policy (lines 7–15)

`COOKIE_MAX_AGE` = 7 days. `COOKIE_OPTIONS`: `httpOnly`, `secure` only in production, `sameSite: 'none'` in production (cross-site, needed when the frontend and backend are on different domains) else `lax`, `path: '/'`.

Block 3 · `setTokenCookie` (lines 17–19)

Writes the `token` cookie with the shared options.

Block 4 · `generateOnboardingNotifications` (lines 21–41)

Creates a `welcome` notification and a `promotion` notification, each only once per user. Promotion text: "🚀 Limited time: 10% off tickets this week!".

Block 5 · `generateToken` (lines 43–45)

Signs `{ id, email }` with 7-day expiry.

Block 6 · `signup` (lines 47–75)

Validates email+password presence and password length ≥ 8 ; 409 on duplicate email; bcrypt (cost 10); creates the user (lowercased email); issues the cookie; fires onboarding notifications in the background; returns `201` with safe user + token.

Block 7 · `login` (lines 77–104)

Looks up the user with `+passwordHash` explicitly selected; uniform 401 for unknown email or wrong password (no user enumeration); issues cookie; returns safe user + token.

Block 8 · `logout` (lines 106–109)

Clears the cookie with the same options.

Block 9 · `me` (lines 111–113)

Echoes `req.user` (already populated by `requireAuth`).

C.9 backend/controllers/userController.js (55 lines)

Wallet linking/unlinking.

Block 1 · Import (line 1)

`User` model.

Block 2 · `isValidEthAddress` (lines 3–5)

Regex `^0x[0-9a-fA-F]{40}$` – format-level validation only (no checksum verification).

Block 3 · `linkWallet` (lines 7–38)

400 if missing or malformed; normalises to lowercase; 409 if another account already has the wallet linked; otherwise updates the user and returns the safe projection.

Block 4 · `unlinkWallet` (lines 40–53)

Sets `walletAddress: null` and returns the updated user.

C.10 backend/controllers/bookingController.js (110 lines)

The most security-sensitive controller: persists bookings only after on-chain verification.

Block 1 · Imports (lines 1–5)

`User`, `Booking`, `Notification`, flight-details generator, and `verifyPurchase`.

Block 2 · `createBooking` – validation (lines 7–19)

Deconstructs the payload; 400 when any of `walletAddress/destinationId/transactionHash/tokenId/pricePaid` is missing.

Block 3 · Wallet match check (lines 21–23)

403 unless the authenticated user's linked wallet equals the booking's wallet (case-insensitive). Prevents booking into a wallet you don't own.

Block 4 · On-chain verification (lines 25–36)

Unless `DISABLE_BOOKING_VERIFY === 'true'`, calls `verifyPurchase` and rejects with `400` on failure, surfacing the specific reason.

Block 5 · Dedupe (lines 38–41)

409 if the transaction hash was already persisted.

Block 6 · Flight fallback (line 44)

`generateFlightDetails(destinationId, { departureDate, travelClass })` fills any fields the client omitted.

Block 7 · Travel-class normalisation (lines 46–51)

Lowercases and whitelists against the enum so a mismatched value cannot fail validation; falls back otherwise.

Block 8 · Persist booking (lines 53–69)

`Booking.create` with lowercased wallet/txHash, per-field fallbacks, default `availabilityStatus: 'confirmed'`.

Block 9 · User tickets append (lines 71–78)

`$push` a denormalised `tickets` entry onto the user document.

Block 10 · Transaction notification (lines 80–87)

Creates a `transaction` notification with tokenId/txHash (fire-and-forget).

Block 11 · Response & error mapping (lines 89–97)

`201` with the booking; catches duplicate-key `11000` as 409.

Block 12 · `getBookingHistory` (lines 99–108)

All bookings for the user, newest first, lean documents.

C.11 backend/controllers/notificationController.js (58 lines)

CRUD-ish notification handlers, all scoped to `req.user.id`.

Block 1 · Import (line 1)

`Notification` model.

Block 2 · `getNotifications` (lines 3–12)

User-scoped list, `createdAt: -1`, limited to 50.

Block 3 · `markRead` (lines 14–24)

Scoped update of one notification by `:id` + owner.

Block 4 · `markAllRead` (lines 26–36)

`updateMany` unread→read for the user.

Block 5 · `dismissPromotion` (lines 38–48)

Scoped to `type: 'promotion'` only — the only type that is dismissible.

Block 6 · `createNotification` (lines 50–57)

Accepts `type, message, tokenId, txHash` and creates a user-scoped notification; `201`.

C.12 backend/controllers/nftController.js (39 lines)

Bridges SVG strings to pinned Pinata image + metadata CIDs.

Block 1 · Import (line 1)

Pinata service helpers.

Block 2 · `uploadTicket` (lines 3–37)

Requires `svgString` (400 otherwise). Uploads the SVG as PNG to Pinata, builds the metadata object (`name` default `SolarExpress Ticket`, `description` default official boarding pass text, `image` as `ipfs://CID`, `external_url` default

`https://solar-express.vercel.app`), attaches `attributes` if an array is supplied, uploads the JSON, and returns `{ success, imageCid, metadataCid, metadataUri, imageUrl }`. Errors are logged and surfaced with `500`.

C.13 backend/services/verifyPurchase.js (75 lines)

Proves a booking corresponds to a real, successful on-chain purchase on Sepolia. Prevents forged bookings.

Block 1 · Imports (lines 1–2)

Viem client helpers and the `sepolia` chain definition.

Block 2 · Sale contract (line 4)

`SALE_CONTRACT` from `SALE_CONTRACT_ADDRESS` env, defaulting to the live `0xA3E410c1A85Ae21774a6aF2D54a818BaedF19eCE`.

Block 3 · RPC fallbacks (lines 6–14)

Seven endpoints: env override first, then publicnode, 1rpc, drpc, tenderly gateway, ethpandaops, publicnode mirror – filtered for empties.

Block 4 · Event ABI (lines 16–18)

`parseAbiItem` of the `TicketPurchased` event signature.

Block 5 · Signature & hash format check (lines 20–25)

Rejects hashes not matching `^0x[a-fA-F0-9]{64}$`.

Block 6 · Client & receipt (lines 27–35)

Public client over a ranked `fallback` transport (15 s per-call timeout). Fetches the receipt; fails if missing or `status !== 'success'`.

Block 7 · Log scan & decode (lines 37–53)

Iterates receipt logs, keeps only those from the sale contract, attempts `decodeEventLog`, and captures the first `TicketPurchased` match. Malformed logs are skipped.

Block 8 · Match check (lines 55–57)

No matching event → failure.

Block 9 · Field verification (lines 59–70)

Compares event args against the submitted values: `buyer` (case-insensitive), `tokenId` and `destinationId` (BigInt equality), and `pricePaid` (`parseEther` of the ETH string).

Block 10 · Export (lines 72–75)

Returns `{ ok: true }` or an `{ ok: false, error }`; exports `verifyPurchase`.

C.14 backend/services/pinataService.js (66 lines)

Pinata/IPFS integration: SVG→PNG upload and metadata JSON upload.

Block 1 · Setup (lines 1–4)

`sharp` for rasterisation; `PINATA_JWT`; API base `https://api.pinata.cloud`; gateway base `IPFS_GATEWAY` default `https://ipfs.io`.

Block 2 · `requireAuth` guard (lines 6–10)

Throws if `PINATA_JWT` is missing (this is why `/api/nft/generate-metadata` returns `503` when it is unset).

Block 3 · `uploadTicketImageToPinata` (lines 12–35)

Sanitises the filename, renders the SVG to PNG with `density: 144`, sends `pinFileToIPFS` as multipart with the JWT, throws with the status/body on failure, returns `IpfsHash`.

Block 4 · `uploadJSONToPinata` (lines 37–56)

JSON payload to `pinJSONToIPFS` with the same auth; returns `IpfsHash`.

Block 5 · URI helpers (lines 58–64)

`ipfsUri(cid)` → `ipfs://<cid>`; `gatewayUrl(cid)` → `<gateway>/ipfs/<cid>`.

Block 6 · Export (line 66)

Exports the four functions.

C.15 `backend/utils/generateFlightDetails.js` (53 lines)

Synthetic flight details generator, mirroring `src/data/travel.ts`.

Block 1 · Doc comment (lines 1–5)

States the fallback purpose and the mirror relationship to the frontend data module.

Block 2 · `LAUNCH_TIMES` (lines 7–11)

Destination id → departure time for all 20 bodies (e.g. 1 → `08:00`, 20 → `00:00`).

Block 3 · `TERMINALS` (lines 13–21)

Destination id → themed terminal name (e.g. 1 → `Mercury Orbital Hub`, 20 → `Pluto Frontier Gate`).

Block 4 · `generateBookingReference` (lines 23–28)

Six chars from an ambiguous-character-free set, prefixed `SX-` (e.g. `SX-K7M2P9`).

Block 5 · `generateFlightDetails` (lines 30–51)

Builds `flightNumber` (`SX###` from destination id), reference, next-day `departureDate` (ISO date), scheduled `departureTime`, terminal, and a random seat (row 1–40, letter A–F). Honors `departureDate` / `passengerClass` overrides.

Block 6 · Export (line 53)

Exports `generateFlightDetails`.

C.16 `backend/scripts/backfillFromContract.js` (172 lines)

One-time reconciliation: reads `TicketPurchased` events from the sale contract and creates any missing MongoDB booking records.

Block 1 · Usage doc (lines 1–11)

Documents invocation and required env (`MONGODB_URI`, `RPC`).

Block 2 · Env & imports (lines 12–17)

Loads `backend/.env` explicitly; mongoose; viem primitives; the flight generator.

Block 3 · Config (lines 19–41)

`MONGO_URI`; `RPC` fallback list; `TICKET_SALE_ADDRESS` from env overrides, defaulting to the live contract; exits if Mongo URI is missing.

Block 4 · Event ABI & client (lines 43–50)

`parseAbiItem` of `TicketPurchased`; a public client over the ranked fallback transport.

Block 5 · `main` — connect & inventory (lines 52–64)

Connects to Mongo, grabs raw `bookings` / `users` collections, and loads the distinct lowercased transaction hashes already in the DB.

Block 6 · Event scan (lines 66–87)

Determines `latestBlock`; scans from `START_BLOCK` (env or `latest - 200000`) in batches of 10000 blocks with a 500 ms throttle between batches; accumulates matching logs.

Block 7 · Block-time cache (lines 89–101)

Caches block timestamps so purchase dates are not re-fetched per log.

Block 8 · Per-log reconciliation (lines 103–163)

Skips hashes already stored; looks up a user by buyer wallet and skips (counted) if none; generates flight details; inserts a `Confirmed` booking with purchase time from the block timestamp; conditionally pushes the ticket onto the user's `tickets` array (guarded by `$ne` tokenId).

Block 9 · Summary & error handling (lines 165–172)

Prints created/skipped/no-user counts, disconnects, and routes fatal errors through `main().catch`.

C.17 backend/scripts/backfillFlightDetails.js (69 lines)

One-time script that generates missing flight fields for existing bookings.

Block 1 · Usage doc (lines 1–11)

Documents scope and env requirements.

Block 2 · Env & imports (lines 12–14)

Loads `backend/.env`; mongoose; flight generator.

Block 3 · Guard (lines 16–20)

Exits if `MONGO_URI` / `MONGODB_URI` unset.

Block 4 · `main` (lines 22–64)

Connects; queries bookings missing any of `flightNumber`, `bookingReference`, `departureDate`, `departureTime`, `launchTerminal`, `seatNumber` (`$or/$exists:false`); streams via cursor; regenerates fields per destination and `$set` s them; logs each update; prints a total.

Block 5 · Error handling (lines 66–69)

`main().catch` exits non-zero on failure.

C.18 backend/package.json (22 lines)

Backend dependency manifest.

Block 1 · Metadata & scripts (lines 1–8)

Name `solarexpress-backend`, version `0.1.0`, private. Scripts: `start` and `dev` both run `node server.js` (no dev server/watch tooling).

Block 2 · Dependencies (lines 9–21)

`bcryptjs ^2.4.3`, `cookie-parser ^1.4.6`, `cors ^2.8.5`, `dotenv ^17.4.2`, `express ^4.21.0`, `express-rate-limit ^8.6.1`, `helmet ^8.3.0`, `jsonwebtoken ^9.0.2`, `mongoose ^9.8.0`, `sharp ^0.35.3`, `viem ^2.55.10`.

Part D — Frontend (Next.js / React)

Blocks are presented in source order and cover every code block. Parts D.1–D.6 are grouped configuration/data/client libraries; D.7–D.12 cover pages, API routes and components. Interactive UI files are described functionally; line ranges are given for the deterministic server/config files.

D.1 Root Configuration

package.json (root)

Frontend manifest.

Block 1 · Scripts

`dev`, `build`, `start`, `lint` standard Next.js 16 scripts.

Block 2 · Dependencies

Framework: `next 16.2.11`, `react 19.2.4`, `react-dom 3D: three ^0.185.1`, `@react-three/fiber ^9.6.1`, `@react-three/drei ^10.7.7`. Chain: `wagmi ^3.7.4`, `viem ^2.55.10`. Server state: `@tanstack/react-query ^5.101.4`. Image tooling: `@resvg/resvg-js ^2.6.2`, `sharp ^0.35.3`. Styling: `tailwindcss ^4`. Language: `typescript ^5`.

tsconfig.json

TypeScript configuration.

Block 1 · Compiler options

`strict` mode, `moduleResolution: bundler`, `jsx: preserve`, `module/ES2022`, and the path alias `@/*` → `./src/*`.

next.config.ts

Next.js configuration; the piece that ties the frontend to the backend.

Block 1 · API rewrites

Rewrites `/api/auth/:path*`, `/api/users/:path*`, `/api/bookings/:path*`, and `/api/notifications/:path*` to `getBackendUrl() || http://localhost:4000`, so the browser only ever talks to the Next origin while backend calls are proxied server-side. (The NFT `/api/nft/*` routes are real Next API routes and are *not* rewritten.)

Block 2 · Server externals

Places `@resvg/resvg-js` in `serverExternalPackages` so the native SVG renderer is loaded correctly by serverless functions.

.gitignore (root)

Ignore rules.

Block 1 · Patterns

Ignores `node_modules`, Next build outputs (`/.next`, `/out`), `.env*` (all env files), local logs, `contractaddresses.txt`, large raw Earth textures, Hardhat/Foundry caches (`.deps`, `artifacts`), `.vercel`, and TS artifacts. Note: `.env.local` is consequently ignored even though it exists locally.

D.2 App Shell

src/app/layout.tsx

Root layout and provider nesting.

Block 1 · Imports & metadata

Imports fonts/globals, providers, and `MobileBottomNav`; exports the app metadata (title/description) for SEO.

Block 2 · RootLayout

Nests providers outside-in: `Web3Provider` (Wagmi + Query) → `AuthProvider` (JWT session) → `LoadingProvider` (pending-tx UI) → `NotificationProvider`. Renders `{children}` and the `MobileBottomNav`.

src/app/globals.css

Global Tailwind v4 stylesheet.

Block 1 · Tailwind import & theme

`@import "tailwindcss";` `:root` and `.dark` CSS variables (HSL palette) for the dark-first design.

Block 2 · Base styles

`html`, `body` use near-black `#09090b` and slate `#f1f5f9` with the design palette; violet `focus-visible` outlines for accessibility.

Block 3 · Utilities

Custom classes `hide-scrollbar`, `scrollbar-thin`, `glass-card` / `glass-card-hover` (frosted panels), and `btn-violet` (primary action button).

src/app/loading.tsx

Global loading boundary.

Block 1 · RootLoading

Renders the animated `LoadingScreen` with the message "Loading SolarExpress..." while routes stream.

D.3 Providers, Context & Client Libs

src/components/providers/Web3Provider.tsx

Wagmi + TanStack Query bootstrap.

Block 1 · Providers

Wraps children in `WagmiProvider` (from `src/lib/wagmi-config.ts`) and `QueryClientProvider` (memoised `QueryClient`).

src/lib/wagmi-config.ts

Wagmi connection configuration.

Block 1 · Config

`createConfig` for the Sepolia chain, an injected connector (MetaMask) with shim disambiguation, and an HTTP transport over `src/lib/rpc.ts`.

src/lib/backend-url.ts

Backend base-URL resolution.

Block 1 · getBackendUrl()

Returns `API_BACKEND_URL` if set, else `http://localhost:4000`. Used by rewrites and client fetch calls.

src/lib/rpc.ts

Sepolia RPC endpoints.

Block 1 · RPC list

Six public Sepolia RPCs ordered by observed reliability, with `NEXT_PUBLIC_SEPOLIA_RPC_URL` appended last so a project override takes precedence (used by the Wagmi transport).

`src/lib/useMobile.ts`

Responsive media-query hook.

Block 1 · useMediaQuery wrapper

Exposes a boolean that tracks a CSS media query (used to branch mobile vs. desktop UI, e.g. `PLANET_ORDER` rendering).

`src/lib/modal-store.ts`

External (non-React) modal store.

Block 1 · Store

Implements `subscribe` / `getIsModalOpen` / `setIsModalOpen` so non-React modules (e.g. `Planet.tsx`) can read modal state via `useSyncExternalStore`.

`src/context/auth-context.tsx`

JWT session management on the client.

Block 1 · Session bootstrap

On mount calls `/api/auth/me` (proxied to the backend) to restore the session; exposes `checkingSession`.

Block 2 · Actions

`signIn` / `signUp` / `signOut` talk to the proxied auth endpoints and update context state on success.

`src/context/loading-context.tsx`

Global "transaction in flight" state.

Block 1 · Pending-tx tracking

Exposes a loading flag (and the pending tx hash from `sessionStorage`) that the app shell uses to show a blocking `LoadingScreen` while a purchase transaction is being mined or synced.

`src/context/notification-context.tsx`

Notification bell state.

Block 1 · Fetch & count

Fetches `/api/notifications` whenever the authenticated user changes; derives `unreadCount`; exposes mark-read/dismiss actions used by the NavBar bell.

D.4 API, ABI & NFT Libs

`src/lib/api.ts`

Backend client helpers and shared types.

Block 1 · Types

Defines `SyncBookingPayload` (walletAddress, destinationId, transactionHash, tokenId, pricePaid, travelClass, etc.), `BookingRecord`, and notification records.

Block 2 · `setAuthToken` / `authFetch`

Maintains a client-side auth token and a fetch wrapper that adds `Authorization: Bearer` + `credentials`, so cookie-less (mobile) clients still authenticate.

Block 3 · `syncBooking`

POSTs the purchase payload to `/api/bookings` after a successful mint.

`src/lib/contract.ts`

Contract addresses & ABIs.

Block 1 · Addresses

Exports `TICKET_SALE_ADDRESS`, `TICKET_NFT_ADDRESS`, `BOOKING_HISTORY_ADDRESS` — each overridable via the corresponding `NEXT_PUBLIC_*` env var, defaulting to the live Sepolia deployments (A.6).

Block 2 · ABIs

`TICKET_SALE_ABI` (including the `constructor` entry, plus `buyTicket`, `destinationPrice`, `setDestinationPrices`) and `TICKET_NFT_ABI` (`mintTicket`, `tickets`, `tokenURI`, `baseTokenURI`).

`src/lib/nft-service.ts`

NFT metadata/IPFS helpers for the gallery.

Block 1 · IPFS resolution

Helpers to convert `ipfs://` URLs to HTTPS gateway URLs (`https://ipfs.io/ipfs/<cid>`) and to fetch token metadata JSON.

`src/lib/nft-ticket-data.ts`

On-chain ticket discovery without an indexer.

Block 1 · Constants

`SECONDS_PER_BLOCK = 12n`, `BLOCK_LOOKBACK_MARGIN = 50000n`, `LOG_WINDOW = 6n` — tuning values for log-scan windowing.

Block 2 · `findMintBlock`

Binary-searches recent blocks for the block in which a given tokenId minted, by scanning `TicketPurchased` logs in rolling windows.

Block 3 · `findTicketData`

Resolves a token's `{ destinationId, timestamp }` from contract state and events — used to reconstruct ticket metadata for the gallery.

`src/lib/generate-nft-svg.ts`

Server-safe SVG ticket renderer.

Block 1 · Star field & layout

`STAR_POSITIONS` scaled by `700/1000` from the full-size design; the `generatePlanetSVG` function emits a 340×540 (or scaled) ticket SVG embedding the procedural planet artwork.

D.5 Hooks

`src/hooks/useBuyTicket.ts`

The purchase engine.

Block 1 · Wagmi primitives

Uses `useWriteContract`, `useWaitForTransactionReceipt`, `useSwitchChain`, `useChainId`, `useAccount`. Guards on chain id and connection.

Block 2 · `buyTicket`

Calls `TicketSale.buyTicket(destinationId, travelClass)` with exact `value` and `gas 500_000n`; persists the hash to `sessionStorage('solarexpress_pending_tx')` before confirmation so a refresh can resume.

Block 3 · Receipt & metadata

On receipt, decodes `TicketPurchased` via `decodeEventLog` for the `tokenId`; calls `/api/nft/generate-metadata` for the PNG/JSON/URI; `warmNftCache` swaps `baseTokenURI` for the `/image/` route URL; returns an Etherscan (Sepolia) link for UI feedback.

src/hooks/useTicketInfo.ts

Token read helpers.

Block 1 · Reads

`useTicketData`, `useOwnerOf`, `useTokenURI` against the NFT contract, all gated on `isConnected && address` to avoid unnecessary RPC calls.

src/hooks/useSyncBooking.ts

Backend reconciliation after purchase.

Block 1 · Sync

POSTs the confirmed purchase to the backend (`syncBooking`) and reports success/failure so the wizard can show the final confirmation state.

src/hooks/useOnChainBookings.ts

On-chain history reader.

Block 1 · Read

Calls `BookingHistory.getBookings(address)` and maps the result to `{ ticketId, destinationId, pricePaid, timestamp }[]`, gated on connection.

src/hooks/useBookingHistory.ts

Server-state wrapper for the history tab.

Block 1 · Query

`useQuery` keyed `booking-history`, `staleTime 15s`, `retry 2 × 3s`, hitting `/api/bookings/history`.

D.6 Static Data

src/data/bodies.ts

The 20 destination catalogue.

Block 1 · BODIES array

20 bodies, ids 1–20, each with name, type, description, radius/color, and price. Prices range `0.001` – `0.120` ETH and are mirrored by the contract's `setDestinationPrices` and by `scripts/mint-test-ticket.js`.

src/data/travel.ts

Flight simulation data.

Block 1 · Tables

`LAUNCH_TIMES`, `TERMINALS`, `DISTANCES_AU` keyed by destination id (mirrors `backend/utils/generateFlightDetails.js`).

Block 2 · Pricing & generation

`getClassPriceEth` (Business = $\times 2.5$ base) and `generateFlightDetails` (frontend twin of the backend generator); `PASSENGER_CLASSES` labels.

src/data/routes.ts

Route topology.

Block 1 · Station sequences

Ordered station arrays per destination (used by `RoutePanel`) with illustrative `distanceAU` values.

src/data/nft-art.ts

Per-destination NFT art configuration.

Block 1 · Texture types & configs

`PlanetTextureType` = `rocky` | `martian` | `gas-giant` | `ice-giant` | `icy` | `volcanic` | `hazy` | `earth` | `cratered`; `artConfigs` maps each body to its texture type and art parameters used by `generate-nft-svg.ts` and `NFTTicket`.

src/data/textures.ts

Procedural canvas texture generation (client-only).

Block 1 · Noise primitives

`hash`, `smoothNoise`, `fbm` (fractal Brownian motion), `hexToRgb`.

Block 2 · Body-type generators

Per-type canvas generators (e.g. `generateGasGiant`, `rocky`, `icy`, `volcanic`, `earth-like`) painting onto a canvas for use as R3F textures.

D.7 App Pages

src/app/page.tsx

Home / explore.

Block 1 · Auth redirect

Redirects to `/signin` when no session exists.

Block 2 · 3D scene

Dynamically imports `SolarSystem` with `ssr: false`, showing a spinner while it loads; wires destination selection to the menu and detail panels.

src/app/signin/page.tsx

Authentication page.

Block 1 · Form

Email/password form with sign-in/sign-up toggle; enforces password ≥ 8 characters; on success redirects to `/`.

`src/app/booking-history/page.tsx`

NFT gallery + booking history.

Block 1 · Tabs & helpers

Two tabs: `nft-gallery` and `booking-history`. Utilities `shorten` (address truncation), `formatDate`, `resolveIPFS` (`ipfs://` → `https://ipfs.io/ipfs/`).

Block 2 · Gallery

For each on-chain booking (`useOnChainBookings`), resolves `tokenURI` via `useTokenURI`, fetches the metadata JSON, and renders a full-size `NFTTicket` in a `Modal` on click.

Block 3 · History

Renders the enriched MongoDB records (flight number, seat, terminal, class, price) from `useBookingHistory`.

`src/app/about/page.tsx`

Project info page.

Block 1 · Static content

Renders `TECH_STACK` and `CONTRACTS` tables (deployed addresses and stack summary).

`src/app/faq/page.tsx`

Frequently asked questions.

Block 1 · Categorized Q&A

Groups of questions with answers (booking, wallets, tickets, network).

D.8 API Routes (NFT metadata/image pipeline)

`src/app/api/nft/generate-metadata/route.tsx`

POST handler that produces IPFS-pinned ticket media right after a mint.

Block 1 · Route config

`maxDuration: 90` (serverless ceiling) and an internal `withTimeout` guard of ~5 s per external call.

Block 2 · Token-id prediction

`predictNextTokenId` scans `TicketPurchased` logs over `RECENT_BLOCK_WINDOW = 60_000n` in `CHUNK_SIZE = 30_000n` batches to infer the just-minted token id.

Block 3 · Media + pinning

Renders the ticket SVG (`generate-nft-svg.ts`), uploads PNG + metadata JSON to Pinata, and — when the signer is the contract owner — calls `setTokenURI` to point the token at `ipfs://CID`. Returns the image/metadata URIs; without `PINATA_JWT` it returns `503`.

`src/app/api/nft/metadata/[tokenId]/route.ts`

GET on-demand ERC-721 metadata (fallback for non-owner mints / baseTokenURI resolution).

Block 1 · Dynamic + cache

`force-dynamic`; an in-memory `Map` caches results for 60 s.

Block 2 · Validation & lookup

Validates `tokenId` against `^[1-9]\d*$`; reads `SolarExpressTicket.tickets` for the destination and `TicketPurchased` for price/date.

Block 3 · Metadata JSON

Returns `{ name, description, image, attributes }` with attributes Destination, Passenger Class, Token ID, Network, Price; `image` is derived from the request origin pointing at `/api/nft/image/[tokenId]`.

src/app/api/nft/image/[tokenId]/route.ts

GET PNG ticket image renderer.

Block 1 · Dynamic + render

`force-dynamic`; renders the ticket SVG to PNG with `Resvg` and the bundled Roboto fonts.

Block 2 · Caching strategy

Serves `Cache-Control`: degraded tickets (unresolved on-chain data, `priceEth === '0'`) are cached 60 s; fully resolved tickets are cached 86400 s.

D.9 UI & Layout Components

src/components/ui/Modal.tsx

Accessible modal primitive.

Block 1 · Portal & escape

Renders via `createPortal`; closes on Escape and on backdrop click; keeps `modal-store` in sync for the 3D scene.

Block 2 · Animation

Applies `animate-modal-fade` / `animate-modal-scale` classes.

src/components/ui/LoadingScreen.tsx

Full-screen animated loader.

Block 1 · Orbit spinner

Animated orbiting-satellite SVG around the `SolarExpressLogo`; respects `prefers-reduced-motion` by disabling the animation.

src/components/ui/SolarExpressLogo.tsx

Brand logo component.

Block 1 · Sizes & artwork

Sizes `sm 24 / md 32 / lg 48 / xl 64`; a 512-viewBox sun-and-orbit mark with gradient fills.

src/components/layout/NavBar.tsx

Desktop header.

Block 1 · Navigation

Four nav items; notification bell with `unreadCount`; profile dropdown with outside-click close.

Block 2 · Wallet controls

Wagmi connect/disconnect buttons and linked-wallet display.

src/components/layout/MobileBottomNav.tsx

Mobile bottom navigation.

Block 1 · Items

Explore / Tickets / FAQ / About glyph items, `sm:hidden`, and hidden on `/signin`.

src/components/wallet/SignInModal.tsx

Authentication modal.

Block 1 · Form

Email/password with sign-in/sign-up toggle; closes via `Modal`; delegates to `auth-context`.

src/components/menu/PlanetMenu.tsx

Desktop destination menu.

Block 1 · Search & expand

Text search, expandable planet → moon grouping, and a favorites filter.

src/components/menu/MobileExploreView.tsx

Mobile destination list.

Block 1 · Ordered list

Renders bodies in `PLANET_ORDER = [1,2,3,5,8,13,16,18,20]` with expandable moons.

D.10 3D Scene Components

src/components/scene/SolarSystem.tsx

Top-level R3F scene.

Block 1 · Layout

Renders `Sun`, orbit rings, and all bodies; `MOON_CONFIG` maps moon ids (`4,6,7,9,10,11,12,14,15,17,19`) to their parent planets.

src/components/scene/Sun.tsx

Procedural sun.

Block 1 · Granulation

Procedural granulation using 3-octave fbm noise; emissive material + point light.

src/components/scene/Earth.tsx

Special-cased Earth (id 3).

Block 1 · Cloud layer

Procedural cloud texture and a multilayer sphere stack (base + clouds + atmosphere) for the home world.

src/components/scene/Planet.tsx

Generic clickable body.

Block 1 · Modal-aware labels

Uses `useSyncExternalStore(modal-store)` to hide drei `Html` labels while any modal is open (prevents z-fighting/overlap).

Block 2 · Selection behaviour

Special-cases Earth (id 3) and scales the focused body to `1.4x`.

`src/components/scene/PlanetRenderer.tsx`

Canvas-texture planet renderer.

Block 1 · Textures

Generates body textures on a canvas via `hash` / `smoothNoise` / `fbm`, applies with `configureTexture`, and supports separate moon vs. planet generators.

D.11 Panel & Menu Components

`src/components/panel/DestinationPanel.tsx`

Selected-destination details.

Block 1 · Info

Color dot, type, description, distance and travel time; no route shown for id 3 (Earth).

`src/components/panel/RoutePanel.tsx`

Route diagram.

Block 1 · Journey tree

Earth → launch terminal → destination rendered as a vertical path; returns `null` for id 3.

`src/components/panel/BookTicketButton.tsx`

The booking wizard.

Block 1 · Wizard steps

Nine steps (`STEP_LABELS`) from travel info through class/passenger selection to payment and final confirmation, with a progress indicator.

Block 2 · Purchase wiring

Drives `useBuyTicket`; on success runs `useSyncBooking` and invalidates the booking-history query.

D.12 NFT Visual Components

`src/components/nft/PlanetRenderer.tsx`

SVG planet artwork for tickets.

Block 1 · Layered art

Glow, base sphere, and shadow layers; texture groups switch on the body's `PlanetTextureType`.

`src/components/nft/NFTTicket.tsx`

The ticket card.

Block 1 · Sizes & gradients

Two variants – `compact` (340×540) and `full` (700×1100). Uses `useId` for unique gradient ids; shared `LOGO_UID = 'nftlogo'` for the watermark logo.

Part E — Appendices

E.1 Operational Scripts

scripts/ipfs-migrate.mjs

Retro-migrates existing tokens to immutable IPFS metadata.

Block 1 · Config

ESM; requires `PINATA_JWT` and `PRIVATE_KEY`; optional `RPC_URL`, `NFT_ADDRESS`, `METADATA_BASE`, `IMAGE_BASE`; defaults the NFT to the live `SolarExpressTicket`.

Block 2 · Per-token migration

For each requested tokenId, fetches the current PNG + metadata, pins both on Pinata, and calls `setTokenURI` (owner-signed) so Etherscan resolves `ipfs://` directly. Usage example from README: `PINATA_JWT=... PRIVATE_KEY=... node scripts/ipfs-migrate.mjs 2 3 5`. Tickets 1–6 were migrated this way.

scripts/wire-with-viem.js

Post-deployment contract wiring.

Block 1 · ABI & helpers

Uses `parseAbi` for `setSaleContract` / `setBaseTokenURI` / `bookingHistory()` / `saleContract()`; `normalizePrivateKey`; a `waitForMined` helper with a 180 s cap.

Block 2 · Execution

Wires each contract's `saleContract` to the deployed `TicketSale` and updates the token's base URI; tolerant of RPC rate limits with retry.

scripts/set-token-uri.js

One-off token URI updater.

Block 1 · Env & call

Reads `NFT_ADDRESS`, `PRIVATE_KEY`, `SEPOLIA_RPC_URL`, `TOKEN_ID`, `METADATA_URI` and issues `setTokenURI`.

scripts/mint-test-ticket.js

Test minting utility.

Block 1 · Prices & mint

A `PRICES` map (1 → 0.004 ... 20 → 0.120 ETH, matching `src/data/bodies.ts`) is pushed via `setDestinationPrices`, then a `buyTicket` is sent for the chosen destination (default 5).

E.2 Repository README

The root `README.md` documents the features, the architecture diagram (reproduced in A.2), install/deploy steps, the environment-variable tables (A.5), and the NFT metadata endpoint behaviour (per-token IPFS URIs, the on-demand fallback route, the 503 behaviour when `PINATA_JWT` is unset, and the retro-migration command). Its "Future improvements" list includes multi-network support, UI ticket transfers, i18n, and automated tests. Two caveats: it references a `contracts/` folder and Hardhat setup that no longer exist in the tree (the contracts now live at the root and are compiled externally), and its folder tree is partially stale.

E.3 Known Limitations & Pending Actions

Item	Status / Note
Deployer private key exposure	Key material surfaced during the session; the deployer account <code>0x79C85b...72</code> must be rotated/moved to a new key. User-side action.
<code>verifyPurchase</code> fix deployment	Commit <code>cba7ef8</code> (buyer/destination/price checks) is not confirmed live on Render — only proxy liveness was verified.
Non-owner mints & IPFS	<code>setTokenURI</code> is <code>onlyOwner</code> , so a purchase signed by a non-owner wallet falls back to the on-demand metadata route. No Hardhat/Foundry toolchain is present to change this.
Legacy unique wallet index	Dropped at boot by <code>config/database.js</code> so one wallet may link to multiple accounts.
Pinata JWT lifetime	Current JWT expires $\approx$ 2027; renewal needed before then or the IPFS mint pipeline returns <code>503</code> .
<code>.env.local</code> OIDC token	A short-lived Vercel OIDC token is present in the local file (git-ignored, expires quickly); not reproduced in this document.
Testing	No automated unit/integration/CI pipeline is present yet (README lists it as future work).
Price drift risk	Prices exist in three places (contract, <code>bodies.ts</code> , <code>mint-test-ticket.js</code>) and must be kept in sync manually.